Tutorial 02: Shader Colors - Teaching Guide

Overview

This tutorial introduces **varying values** - one of the most important concepts in shader programming. You'll learn how data flows between vertex and fragment shaders, and how the GPU automatically interpolates values across triangles.

Prerequisites and Setup

Installation

Make sure you have completed the setup from Tutorial 01 and have the virtual environment activated:

```
# Activate virtual environment (if not already active)
.\shader_tutorial_venv\Scripts\Activate.ps1

# Navigate to tutorials folder
cd tutorials

# Run tutorial 02
python 02_shader_colors.py
```

Expected Output: A window showing colorful triangles with smooth color gradients blending between vertices on a white background.

What's Different from Tutorial 01:

- Multiple colors instead of solid green
- Smooth color transitions across triangle surfaces
- More complex VBO data (position + color packed together)

Learning Objectives

By the end of this tutorial, students will understand:

1. What "varying" values are and why they're important
2. How to pass data from vertex shader to fragment shader
3. How GPU interpolation works across triangle surfaces
4. How to pack multiple data types (position + color) into a single VBO
5. How to use strides and offsets to read packed data
6. How to handle shader compilation errors

What's New in This Tutorial

Compared to Tutorial 01, we're adding:

- ☑ **Varying variables** for shader communication
- ☑ **Color data** in our VBO (was position-only before)
- ☑ **Color arrays** enabled alongside vertex arrays
- ☑ **Stride-based array access** (reading from packed data)
- ☑ **Error handling** for shader compilation

Code Breakdown

1. Error Handling Demo

```
try:
    shaders.compileShader( """ void main() { """, GL_VERTEX_SHADER )
except (GLError, RuntimeError) as err:
    print('Example of shader compile error:', err)
else:
    raise RuntimeError( """Didn't catch compilation error!""" )
```

What it does:

- Deliberately tries to compile broken shader code
- Shows how errors are reported
- Educational: demonstrates what happens when shaders fail

Common Shader Errors:

1. **Syntax errors:** Missing semicolons, brackets
2. **Type mismatches:** Assigning vec3 to vec4
3. **Undefined variables:** Using variables not declared
4. **Version incompatibility:** Using features not in your GLSL version

Error Message Structure:

```
RuntimeError: ('Shader compilation failed',
               'ERROR: 0:1: '' : syntax error',
               'shader source code here...')
```

---

2. Vertex Shader with Varying

```
varying vec4 vertex_color;
void main() {
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
```

```
    vertex_color = gl_Color;
}
```

New Concept: The `varying` Keyword

What is a varying?

- A variable that connects vertex shader → fragment shader
- Declared **outside** main() in both shaders
- Automatically **interpolated** across the triangle surface

Line-by-line:

1. `varying vec4 vertex_color;` - Declares the varying (global scope)
2. `gl_Position = ...` - Same as before, transforms vertex
3. `vertex_color = gl_Color;` - Copies built-in color to our varying

The Flow:

```
Vertex Shader (runs 3 times per triangle)
    v1: vertex_color = red    (1, 0, 0)
    v2: vertex_color = green  (0, 1, 0)
    v3: vertex_color = blue   (0, 0, 1)
            ↓
    GPU INTERPOLATION MAGIC
            ↓
Fragment Shader (runs once per pixel)
    pixel at center: vertex_color = (0.33, 0.33, 0.33) - greyish
    pixel near v1:   vertex_color = (0.8, 0.1, 0.1)    - reddish
    pixel near v2:   vertex_color = (0.1, 0.8, 0.1)    - greenish
```

---

3. Fragment Shader with Varying

```
varying vec4 vertex_color;
void main() {
    gl_FragColor = vertex_color;
}
```

Key Points:

- Must declare the same varying with **same name and type**
- Each fragment receives an **interpolated** value
- We just pass it through to the final color

What Interpolation Means:

Imagine a triangle with these vertex colors:

```
        Green (0,1,0)
        /\
       /  \
      /    \
     /      \
    /________\
Red           Blue
(1,0,0)    (0,0,1)
```

Fragments in the middle get **blended** colors:

- Center of triangle: ~(0.33, 0.33, 0.33) - grey
- Closer to red corner: more red in the mix
- Closer to blue corner: more blue in the mix

The Math (Barycentric Interpolation):

```
fragment_color = w1*v1_color + w2*v2_color + w3*v3_color

where:
  w1 + w2 + w3 = 1  (weights sum to 1)
  w1, w2, w3 are based on distance from each vertex
```

The GPU does this **automatically** for all varying values!

---

4. Packed VBO Data

```
self.vbo = vbo.VBO(
    array( [
        [  0, 1, 0,  0,1,0 ],  # x, y, z, r, g, b
        [ -1,-1, 0,  1,1,0 ],
        [  1,-1, 0,  0,1,1 ],
        # ... more vertices
    ],'f')
)
```

Memory Layout:

Each row is one vertex record. Each record has 6 floats:

```
Byte Offset:  0   4   8   12  16  20
          [  x,  y,  z,  r,  g,  b ]
```

```
                └──────┬──────┘   └──────┬──────┘
                   Position          Color
```

Why Pack Data?

1. **Cache Efficiency:** GPU reads all data for one vertex in one memory fetch
2. **Modern Standard:** Current best practice for GPU programming
3. **Flexibility:** Can mix different data types

Alternative (not recommended):

```
# Old way: separate arrays
positions = [ [0,1,0], [-1,-1,0], ... ]
colors = [ [0,1,0], [1,1,0], ... ]
```

This requires two separate VBOs and more complexity.

---

5. Color Array Breakdown

Let's examine the actual colors:

```
[  0, 1, 0,  0,1,0 ],  # Vertex 0: Position (0,1,0),   Color Green (0,1,0)
[ -1,-1, 0,  1,1,0 ],  # Vertex 1: Position (-1,-1,0), Color Yellow (1,1,0)
[  1,-1, 0,  0,1,1 ],  # Vertex 2: Position (1,-1,0),  Color Cyan (0,1,1)
```

Triangle 1 Colors:

- Top vertex: **Green** (0, 1, 0)
- Bottom-left: **Yellow** (1, 1, 0) - Red + Green
- Bottom-right: **Cyan** (0, 1, 1) - Green + Blue

Visual Result:

```
        Green
        /\
       /  \
      /    \  (blended colors in middle)
     /      \
    /________\
Yellow      Cyan
```

---

6. Enabling Arrays with Strides

```
glEnableClientState(GL_VERTEX_ARRAY)
glEnableClientState(GL_COLOR_ARRAY)
```

What it does:

- Tells OpenGL we're providing TWO types of data
- `GL_VERTEX_ARRAY` → positions
- `GL_COLOR_ARRAY` → colors

Important: Must enable BEFORE setting pointers!

---

7. Setting Up Vertex Pointer with Stride

```
glVertexPointer(3, GL_FLOAT, 24, self.vbo )
```

Parameters Explained:

1. `3` - Number of components per vertex
    - We have x, y, z (3 values)
2. `GL_FLOAT` - Data type of each component
    - 32-bit floating point
3. `24` - Stride in bytes
    - Distance from start of one vertex record to start of next
    - Calculation: 6 floats × 4 bytes/float = 24 bytes
4. `self.vbo` - Pointer to data
    - Actually a NULL pointer since VBO is bound
    - OpenGL reads from bound VBO starting at offset 0

Visual of Stride:

```
Memory:  [x1 y1 z1 r1 g1 b1][x2 y2 z2 r2 g2 b2][x3 y3 z3 r3 g3 b3]
          ^                 ^
          Read here         Skip 24 bytes to here
          └─────────stride──────────→
```

---

8. Setting Up Color Pointer with Offset

```
glColorPointer(3, GL_FLOAT, 24, self.vbo+12 )
```

Parameters Explained:

1. `3` - Number of components per color
    - We have r, g, b (3 values)
2. `GL_FLOAT` - Data type
3. `24` - Stride in bytes (same as vertex pointer)
4. `self.vbo+12` - Pointer with OFFSET
    - Start reading 12 bytes into the VBO
    - 12 = 3 floats × 4 bytes/float
    - This skips past the position data

Visual of Offset:

```
Memory:  [x1 y1 z1 r1 g1 b1][x2 y2 z2 r2 g2 b2]
          ^        ^         ^        ^
        pos=0   color=12   pos=24   color=36
```

Why +12?

- Position uses 3 floats = 12 bytes
- Color data starts AFTER position data
- So we skip the first 12 bytes

---

9. The Complete Data Flow

In Memory:

```
VBO: [x y z r g b][x y z r g b][x y z r g b]...
      └─┬─┘ └─┬─┘
        │     └─ glColorPointer reads from offset 12
        └─ glVertexPointer reads from offset 0
```

During Drawing:

```
For each vertex:
  1. Read position: bytes [0,1,2] → gl_Vertex
  2. Read color:    bytes [3,4,5] → gl_Color
  3. Run vertex shader
```

```
   4. vertex_color = gl_Color
   5. Store gl_Position and vertex_color
```

After All Vertices:

```
   6. Rasterize triangles
   7. For each fragment:
      - Interpolate vertex_color from 3 vertices
      - Run fragment shader
      - Output final color
```

Key Concepts Explained

1. Varying Variables

What they do:

- Transfer data from vertex shader to fragment shader
- Automatically interpolated across triangles
- Perspective-correct (accounts for 3D depth)

Common Uses:

- Colors (like we're doing)
- Texture coordinates
- Normal vectors for lighting
- Custom data for special effects

Limitations:

- Must be same type in both shaders
- Must have same name
- Limited number available (varies by GPU)

---

2. Interpolation

Linear Interpolation (1D):

```
v1 = 0, v2 = 10
At 50% between: result = 5
At 25% between: result = 2.5
```

Triangular Interpolation (2D):

```
      v1
      /\
     /  \  ← point here is weighted average of v1, v2, v3
    /____\
   v2    v3
```

The GPU handles:

- Finding correct weights
- Performing weighted sum
- Handling perspective distortion
- Clipping (when triangles go off-screen)

---

3. Stride and Offset

Stride:

- How many bytes to skip to get to next record
- Allows interleaved data

Offset:

- Where to start reading within each record
- Allows one VBO to hold multiple data types

Example Layout:

```
struct Vertex {
    float position[3];  // 12 bytes, offset = 0
    float color[3];     // 12 bytes, offset = 12
    // stride = 24 bytes total
};
```

---

4. Built-in Variables Used

In Vertex Shader:

- `gl_Vertex` - Input vertex position (vec4)
- `gl_Color` - Input vertex color (vec4)
- `gl_Position` - Output position (REQUIRED)

In Fragment Shader:

- `gl_Color` - Interpolated color from gl_FrontColor
- `gl_FragColor` - Output pixel color (REQUIRED)

Common Student Questions

Q: Why use "varying" instead of just setting gl_Color directly?
A: For this tutorial, we could! But `varying` is more flexible:

- You can create multiple varying values
- You can name them descriptively
- You can do calculations on them
- Modern GLSL requires custom varying (gl_Color is legacy)

Q: What happens if I forget to declare the varying in the fragment shader?
A: Compilation error! Both shaders must declare matching varying with same name and type.

Q: Can I make a varying without interpolation?
A: Yes! Use `flat varying` in modern GLSL. All fragments in a triangle get the same value (from the "provoking vertex").

Q: Why is my color data stride 24, not 12?
A: Stride is the FULL record size (position + color), not just the color portion.

- Position: 3 floats = 12 bytes
- Color: 3 floats = 12 bytes
- Total stride = 24 bytes

Q: What if I want 4-component colors (RGBA)?
A: Change:

- VBO: add 4th component to each color
- glColorPointer: first parameter becomes 4
- Stride: becomes 28 bytes (3 pos + 4 color = 7 floats × 4)

Q: Can I have multiple varying values?
A: Yes! Example:

```
varying vec4 vertex_color;
varying vec2 texture_coord;
varying vec3 normal;
```

Visualization of Interpolation

Triangle with 3 Different Colors:

```
Input (Vertices):
        Red(1,0,0)
        /\
       /  \
      /    \
```

```
     /______\
Green       Blue
(0,1,0)   (0,0,1)

Output (Fragments after interpolation):
        Red
        /\
       /R \      R = reddish colors
      / RG \     G = greenish colors
     /______\    B = bluish colors
Green  GB  Blue  RG, GB, RGB = blends

Center of triangle ≈ grey (0.33, 0.33, 0.33)
```

What the GPU does:

1. Gets three vertex colors from vertex shader
2. For each pixel inside triangle:
    - Calculate weights based on position
    - Mix three colors using weights
    - Pass to fragment shader

Experiments to Try

1. Change Colors

```
# Make all vertices the same color
[  0, 1, 0,  1,0,0 ],  # All red
[ -1,-1, 0,  1,0,0 ],
[  1,-1, 0,  1,0,0 ],
```

Result: Solid red triangle (no gradient)

2. Black and White Gradient

```
# Create greyscale gradient
[  0, 1, 0,  1,1,1 ],  # White
[ -1,-1, 0,  0,0,0 ],  # Black
[  1,-1, 0,  0,0,0 ],  # Black
```

Result: Fades from white at top to black at bottom

3. Add Transparency

```
// In fragment shader, modify alpha
void main() {
    gl_FragColor = vec4(vertex_color.rgb, 0.5);  // 50% transparent
}
```

4. Modify Color in Fragment Shader

```
// Make everything darker
void main() {
    gl_FragColor = vertex_color * 0.5;
}
```

5. Swap Red and Blue

```
// In fragment shader
void main() {
    gl_FragColor = vertex_color.bgra;  // Swizzle!
}
```

Debugging Tips

Problem: All triangles are one color

- Check that color array is enabled
- Verify stride and offset calculations
- Make sure varying is declared in both shaders

Problem: Weird colors/flickering

- Stride might be wrong (should be 24)
- Offset might be wrong (should be 12)
- Data type might be wrong (should be GL_FLOAT)

Problem: Nothing draws

- Forgot to enable GL_COLOR_ARRAY?
- Vertex shader syntax error?
- VBO data malformed?

Problem: Compilation errors

- Check varying name matches exactly
- Verify types match (vec4 = vec4, not vec3)
- Look for missing semicolons

Debugging Technique: Print Colors

```
// Temporary debug fragment shader
void main() {
    // Output color components separately to see what you're getting
    gl_FragColor = vec4(vertex_color.r, 0, 0, 1);  // Only red channel
}
```

Performance Notes

Why Packed Data is Faster:

1. **Cache Locality:** All data for one vertex is together
2. **Memory Bandwidth:** Fewer memory accesses
3. **GPU Architecture:** Designed for this pattern

Memory Access Pattern:

```
Packed (GOOD):     [pos|col][pos|col][pos|col]  ← One cache line
Separate (BAD):    [pos][pos][pos]...[col][col][col]  ← Two cache lines
```

Best Practices:

- Pack tightly (no wasted space)
- Align to 4-byte boundaries
- Use appropriate types (don't use double if float is enough)

Next Steps

In Tutorial 03, we'll learn:

- **Uniform values** - sending data from CPU to shader
- Doing calculations in the vertex shader
- Creating fog effects based on depth
- Using built-in GLSL functions

Terms Glossary

Varying:
A variable that passes from vertex shader to fragment shader with automatic interpolation

Stride:
Number of bytes between the start of consecutive vertex records

Offset:
Number of bytes from the start of a record to a specific attribute

Interpolation:
Blending values smoothly across a triangle surface

Packed Data:
Multiple attributes stored together in one array

Barycentric Coordinates:
The math behind how GPU determines interpolation weights (students don't need to understand this in detail)

Rasterization:
The process of converting triangles into fragments

Fragment:
A potential pixel (may be discarded or overdrawn)

Visual Summary

```
Python:     VBO with packed data [pos|color][pos|color]...
                        ↓
                   glVertexPointer(stride=24, offset=0)
                   glColorPointer(stride=24, offset=12)
                        ↓
Vertex Shader:  gl_Vertex (position) + gl_Color (color)
                        ↓
                vertex_color = gl_Color  (varying output)
                        ↓
GPU:            Interpolate across triangle
                        ↓
Fragment Shader: vertex_color (interpolated)
                        ↓
                gl_FragColor = vertex_color
                        ↓
Screen:         Colored triangle with smooth gradients!
```

This is the foundation of all modern shader-based graphics!